

Experiment 1: Rogue One

In this experiment we wish to explore what happens with the decision boundary depending on where the point to forget is positioned (for the synthetic data we call this the "rouge point"). We experiment with placing it in a wrong class, far away from the centroid of its class, etc.

See [Experiment 1: Rogue Many](#) for results with a cluster of rogue points.

[!NOTE]

The Amnesiac unlearning method does not have a repair step. Might not be necessary for this experiment as we are forgetting a single point and points are easily separable.

[!NOTE]

The SAE model is slightly different from the other models (it uses skip connections) and the results before unlearning will be different than the others, even when using the same seed. The same is the case for the SISA model as it is an ensemble of models.

How to use

To generate the data, run:

```
python exp1_rouge_one/generate_data.py
```

To run the experiment for all the datasets and unlearning methods, run:

```
python exp1_rouge_one/experiment_run.py
data.dataset="data/data_1.npz,data/data_2.npz,data/data_3.npz,data/data_4.
npz,data/data_5.npz" unlearn.method="amnesiac,ssd,retrain,scrubr,sae,sisa"
hydra/launcher=ray --multirun
```

[!TIP] The `hydra/launcher=ray` argument will run the experiments in parallel using Ray.

You can overwrite any parameters from the config file, for example `n_epochs`:

```
python exp1_rouge_one/experiment_run.py trainer.n_epochs=10
```

The network

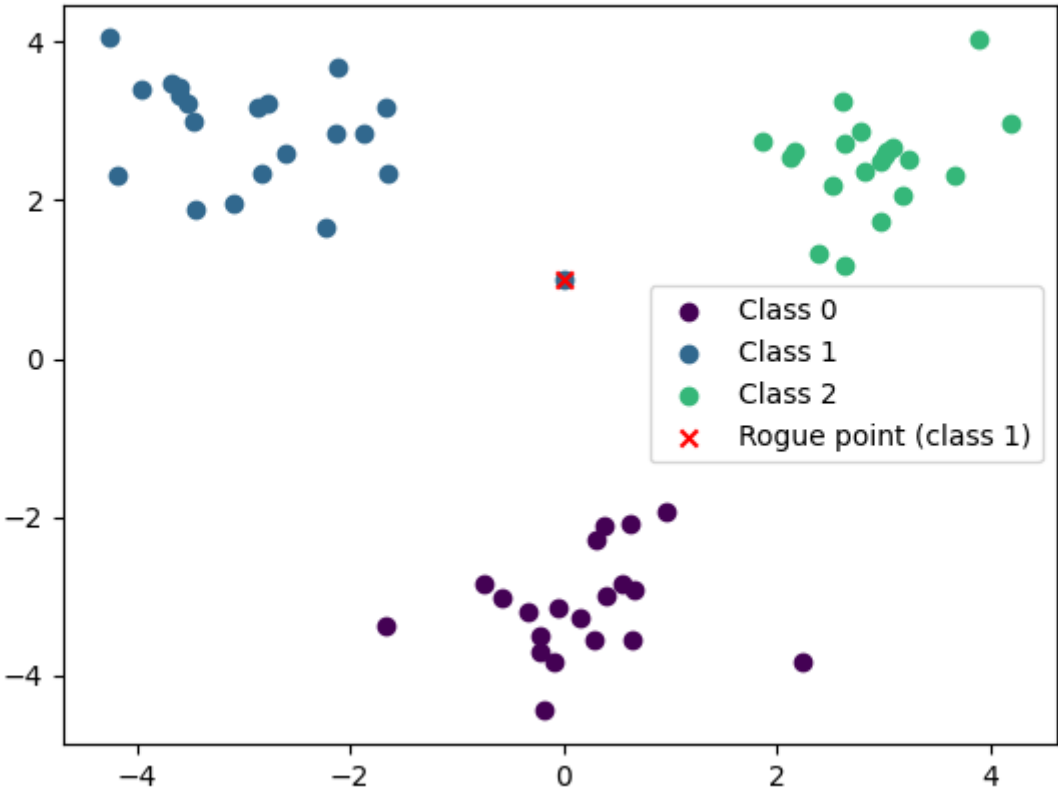
The network is a simple feedforward network which can be seen below:



Experiment runs

Now we run the experiment on each of the dataset configuration, moving the rogue point.

- 1. Rogue point with same distance to all centroids (close to decision boundary)

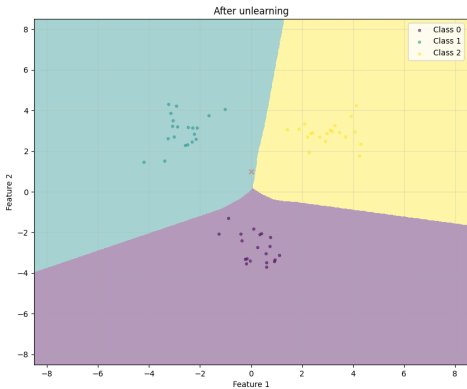
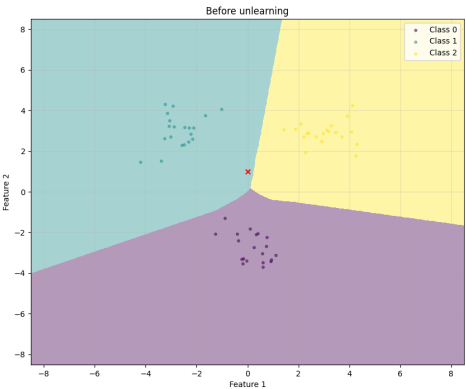


Findings

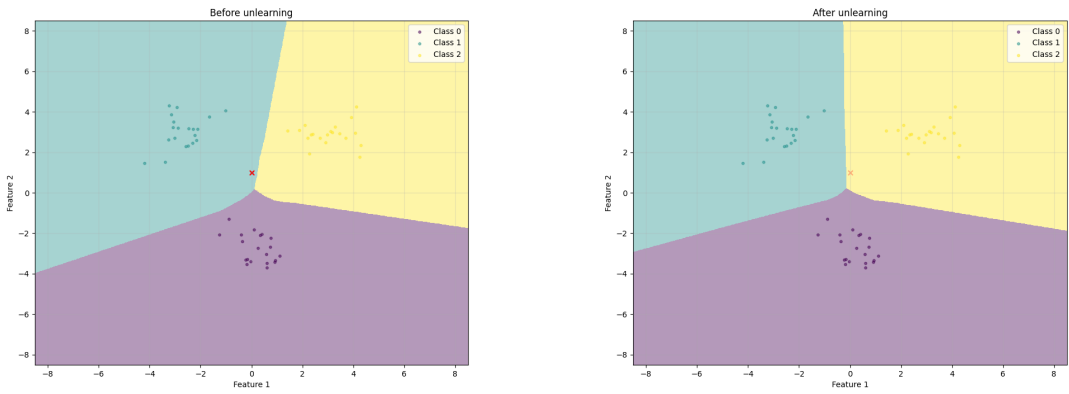
Before unlearning

After unlearning

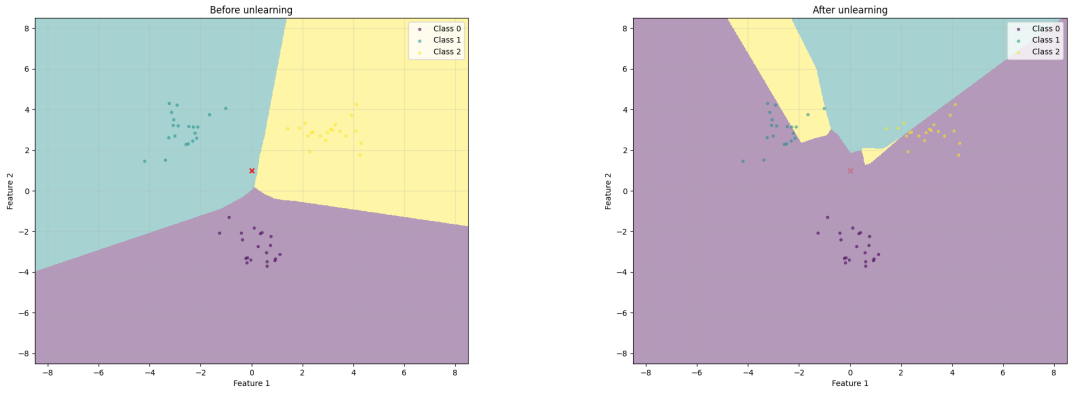
Retrain



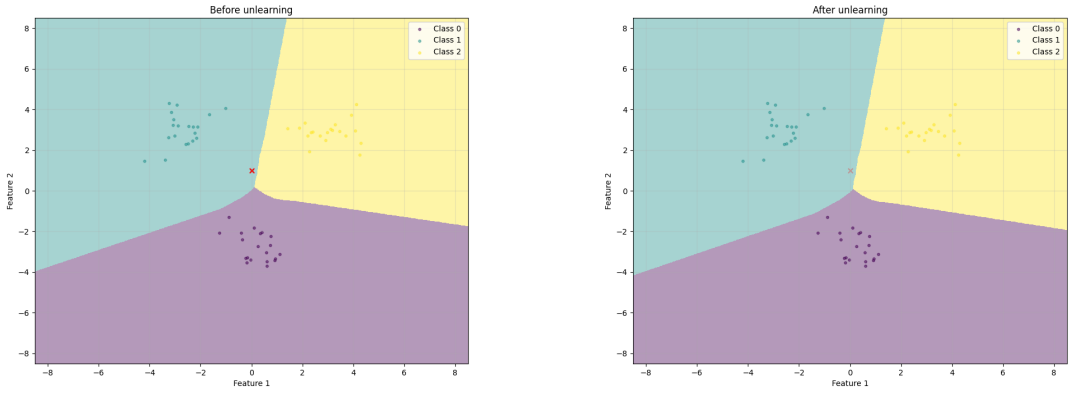
Scrub+R



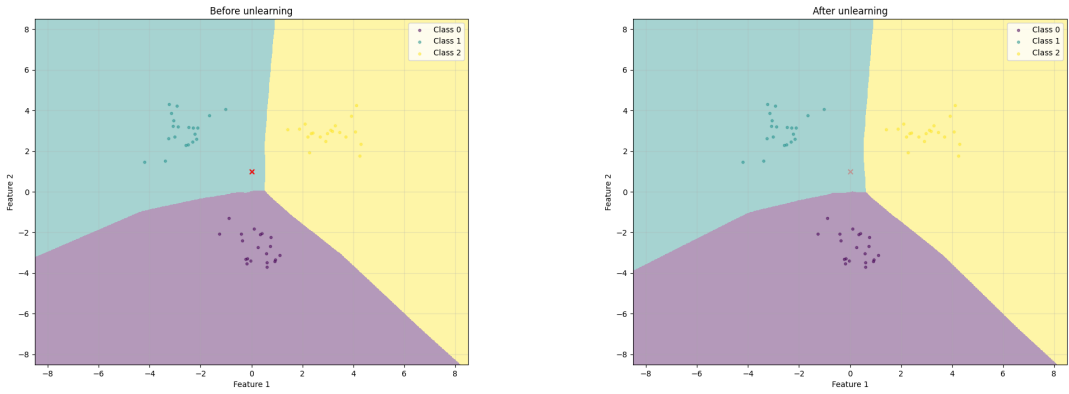
SSD



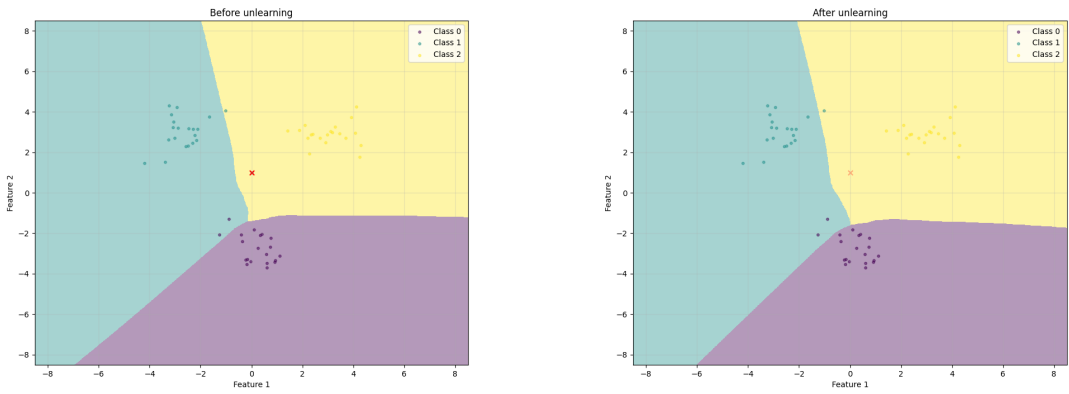
Amnesiac



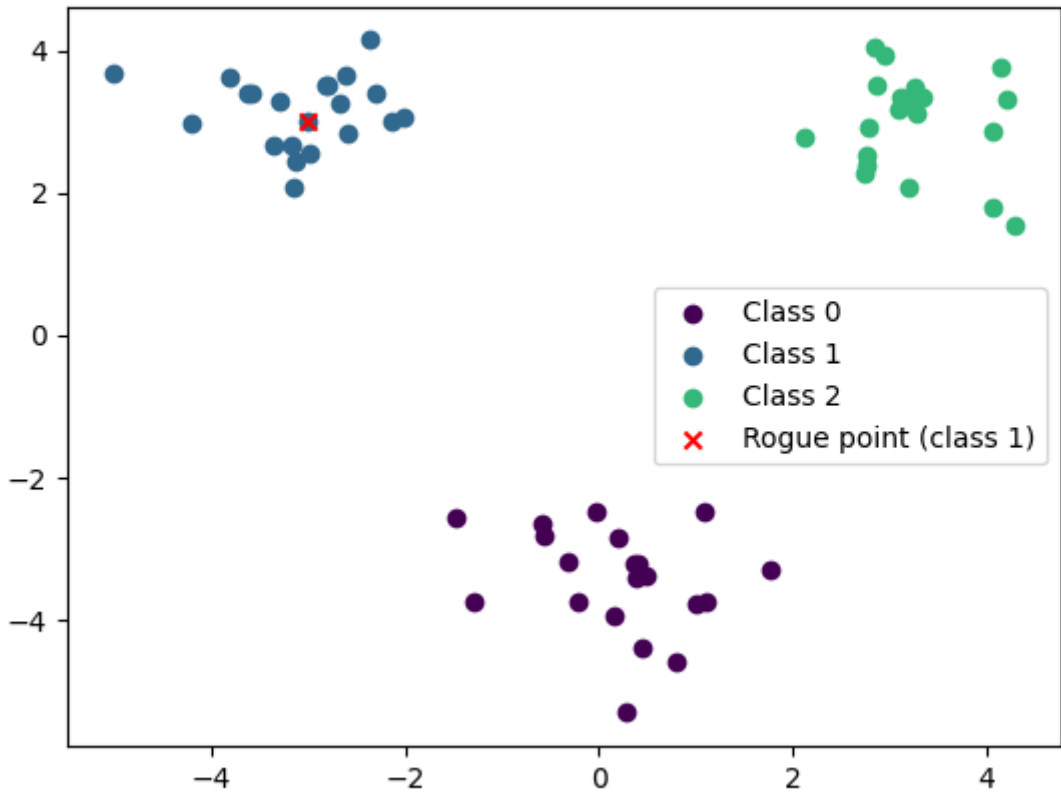
SAE



SISA

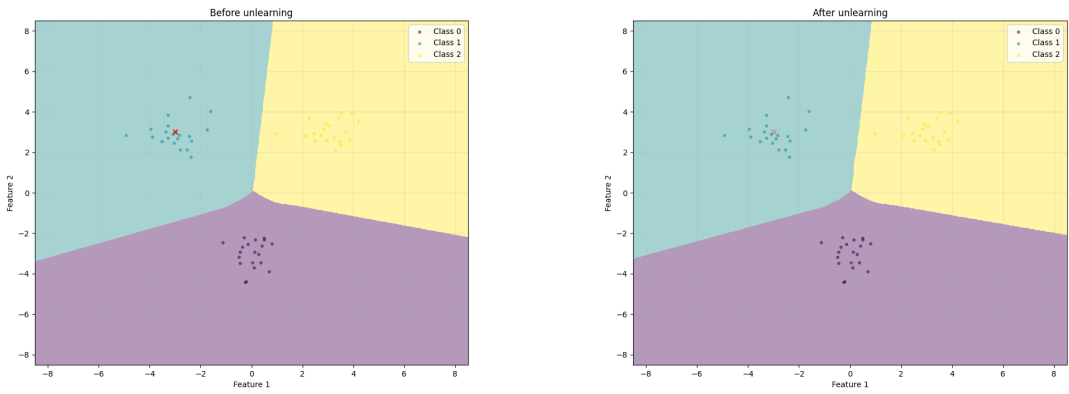


2. Rogue point with same centroid as its class

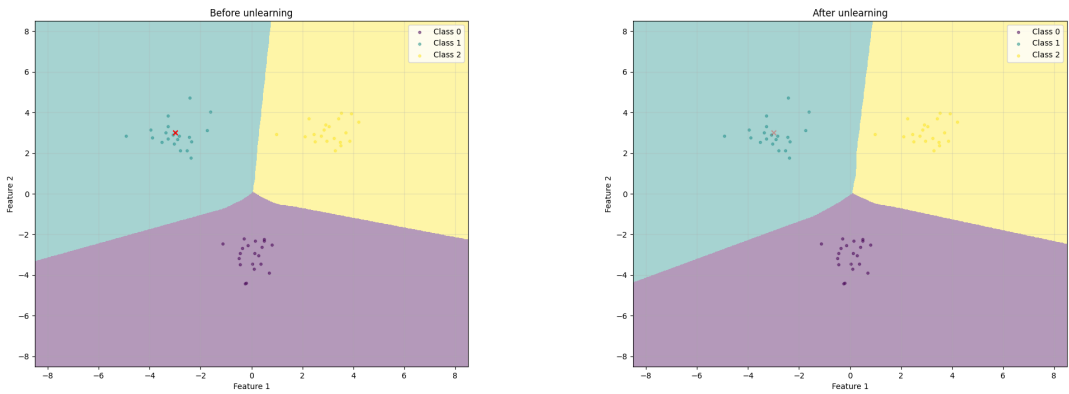


Findings

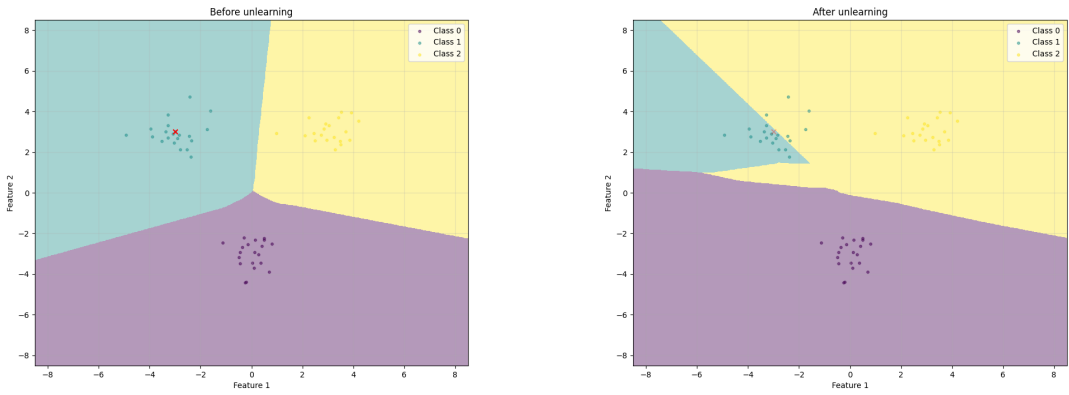
	Before unlearning	After unlearning
Retrain		



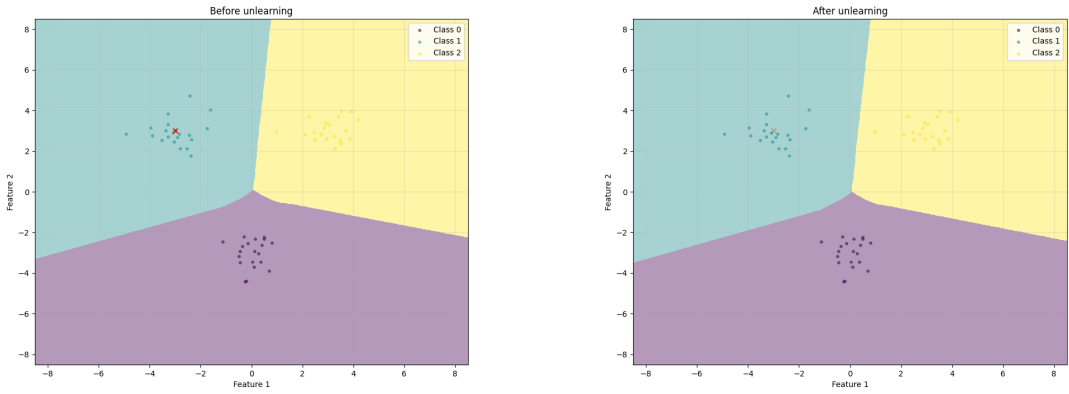
Scrub+R



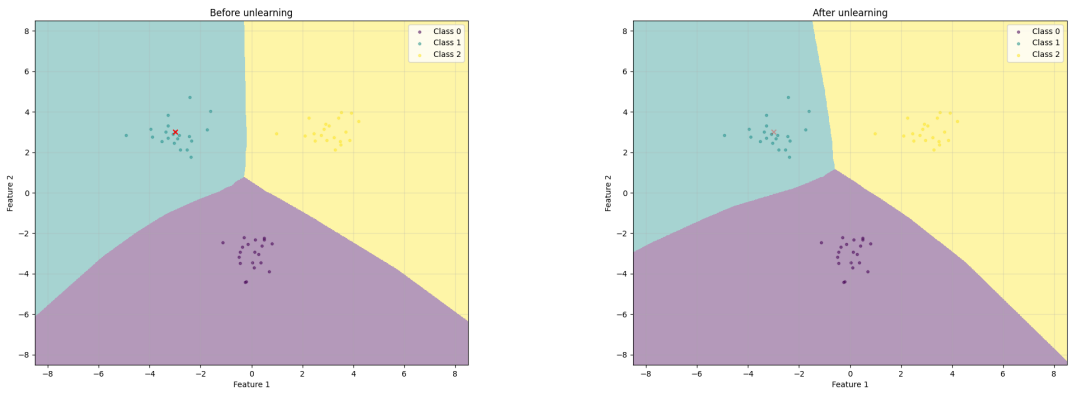
SSD



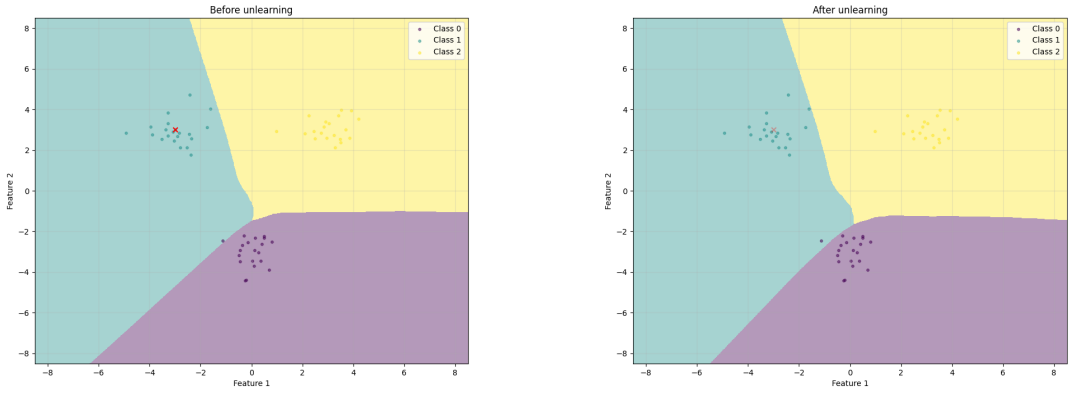
Amnesiac



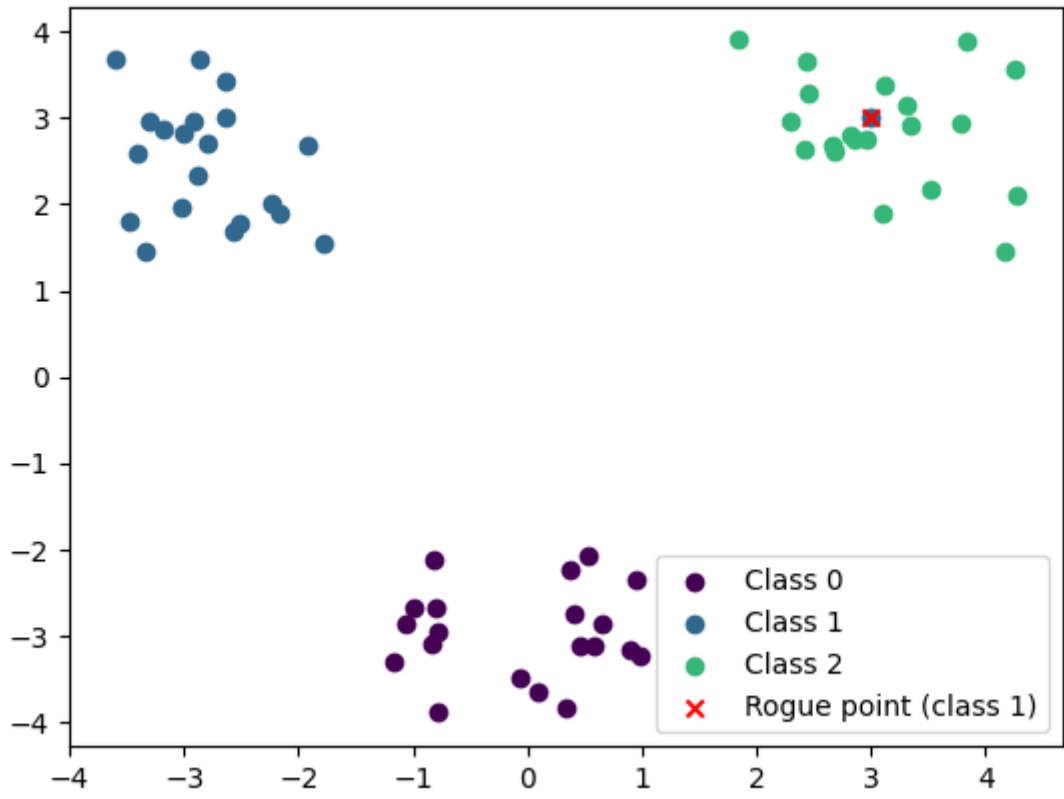
SAE



SISA



3. Rogue point with same centroid as the class with a different label

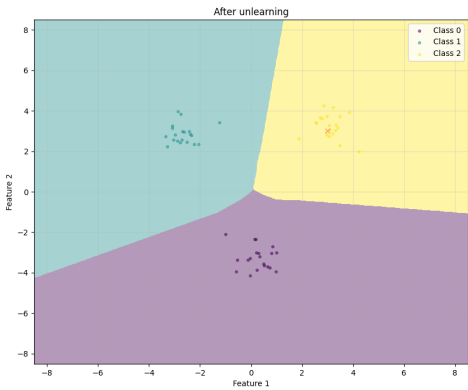
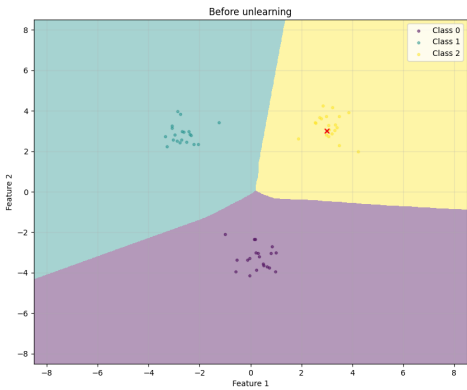


Findings

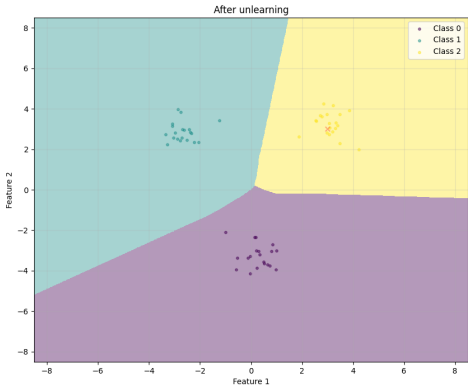
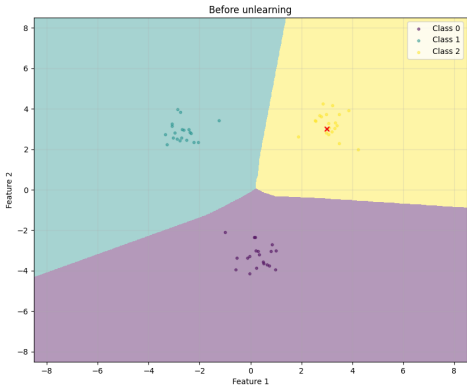
Before unlearning

After unlearning

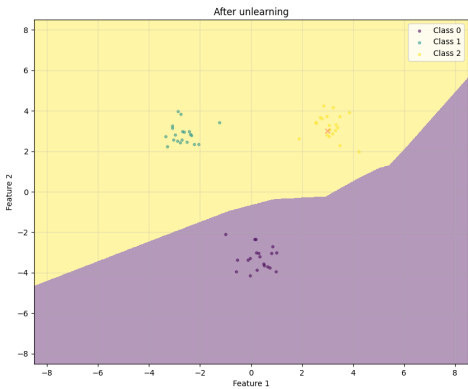
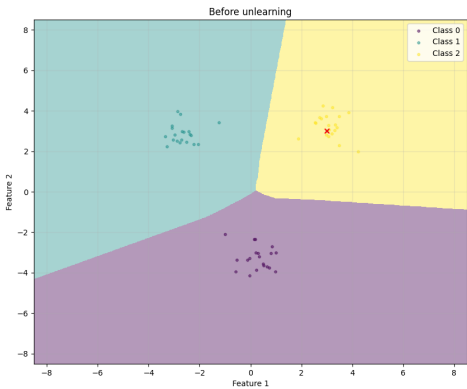
Retrain



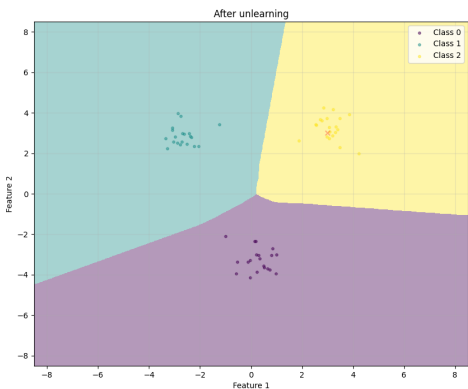
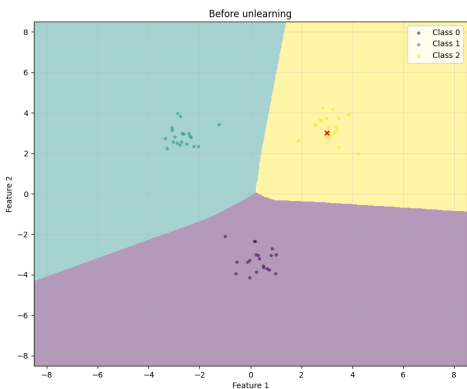
Scrub+R



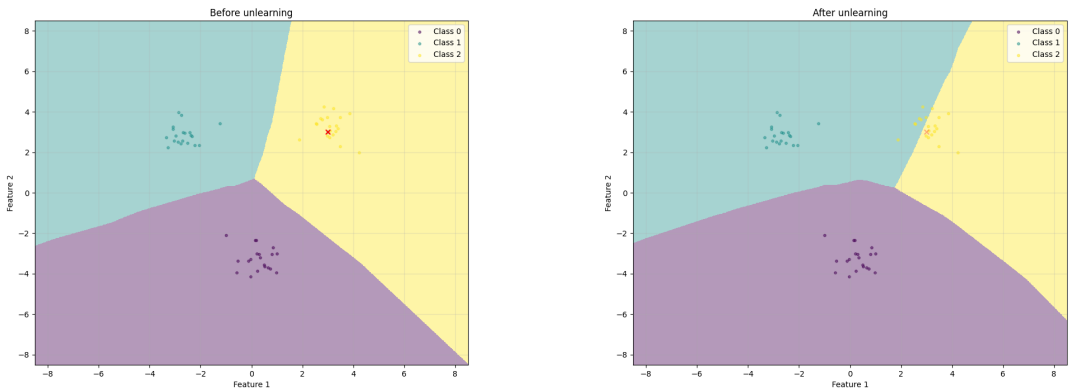
SSD



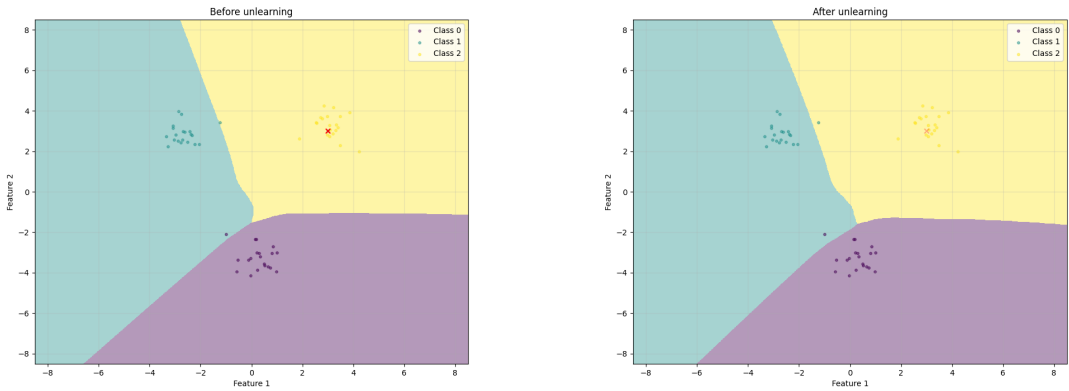
Amnesiac



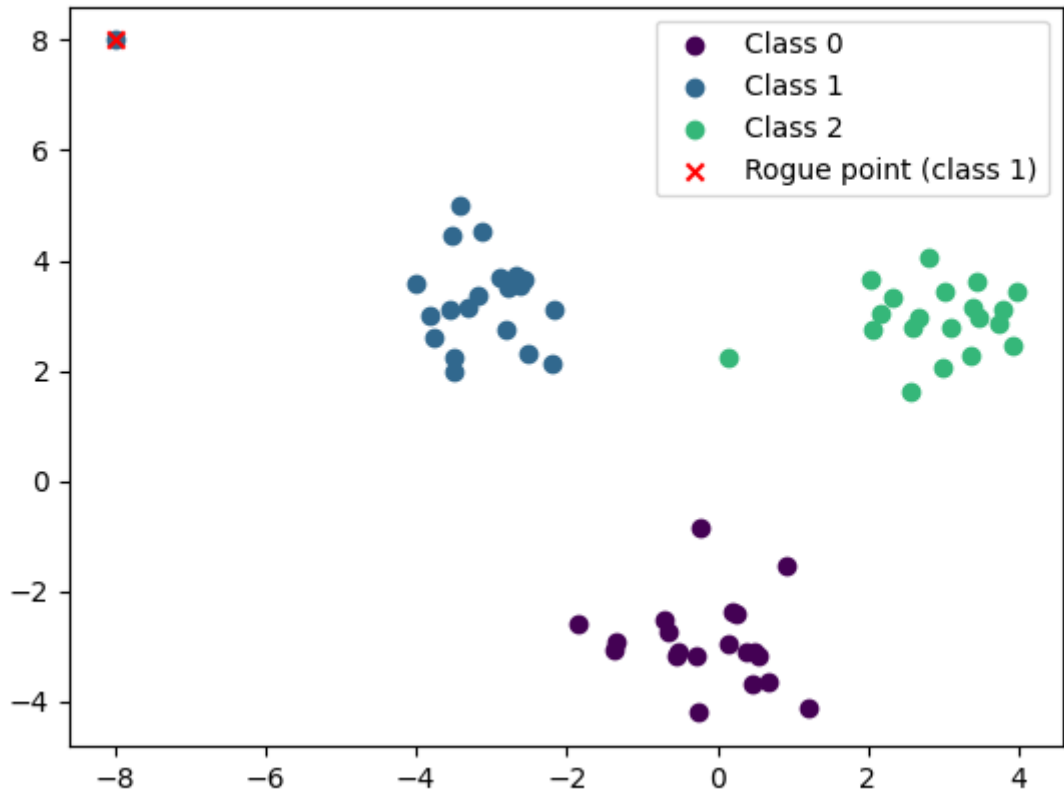
SAE



SISA



4. Rogue point far away from its centroid, but probably in the same decision boundary

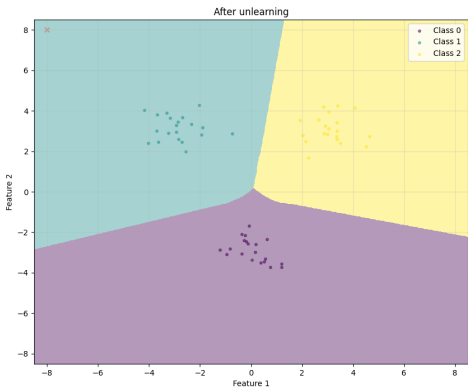
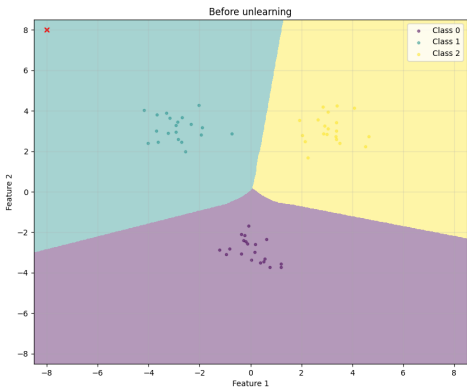


Findings

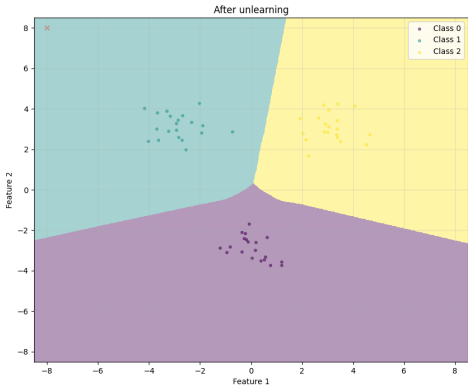
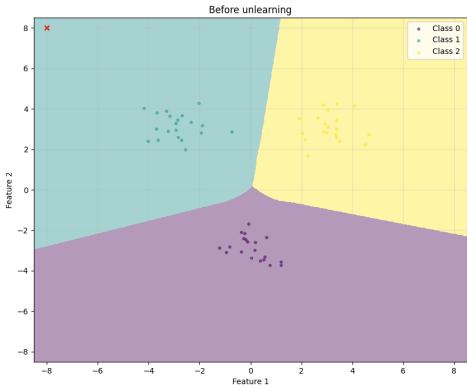
Before unlearning

After unlearning

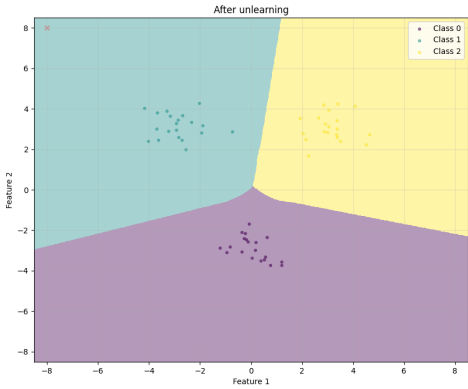
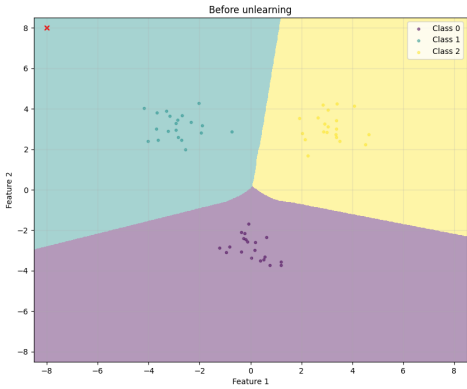
Retrain



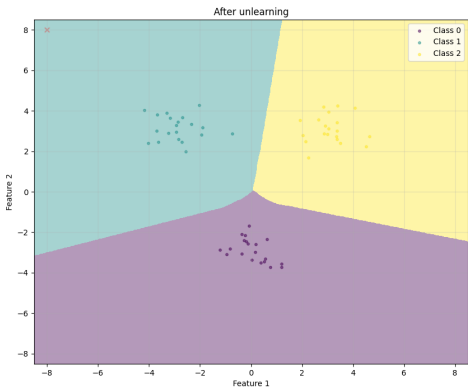
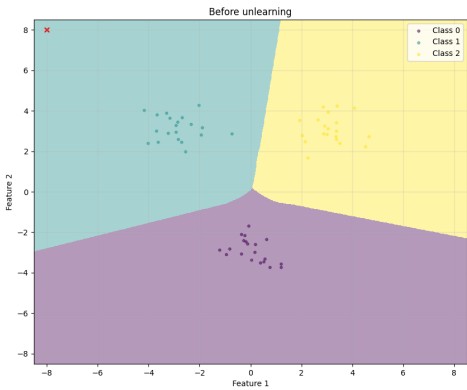
Scrub+R



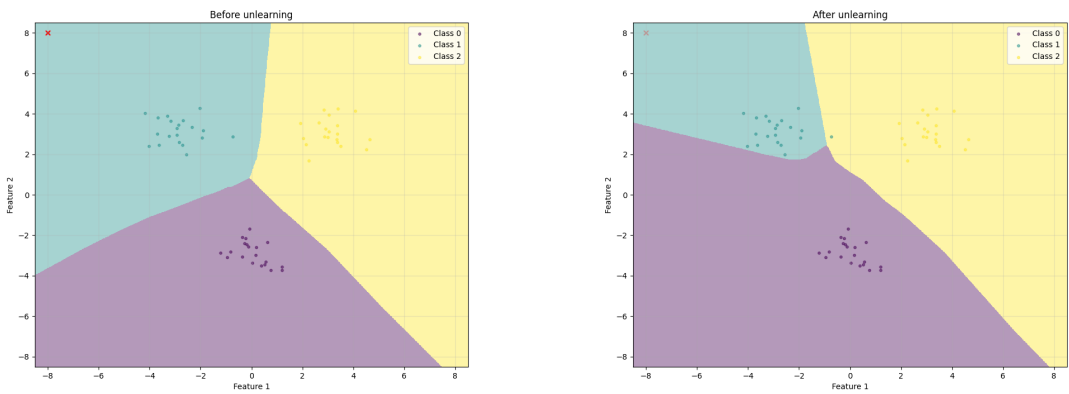
SSD



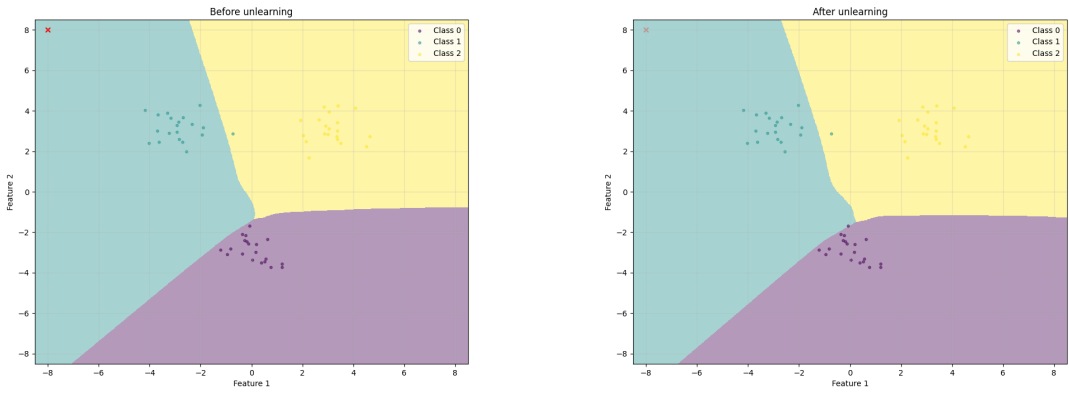
Amnesiac



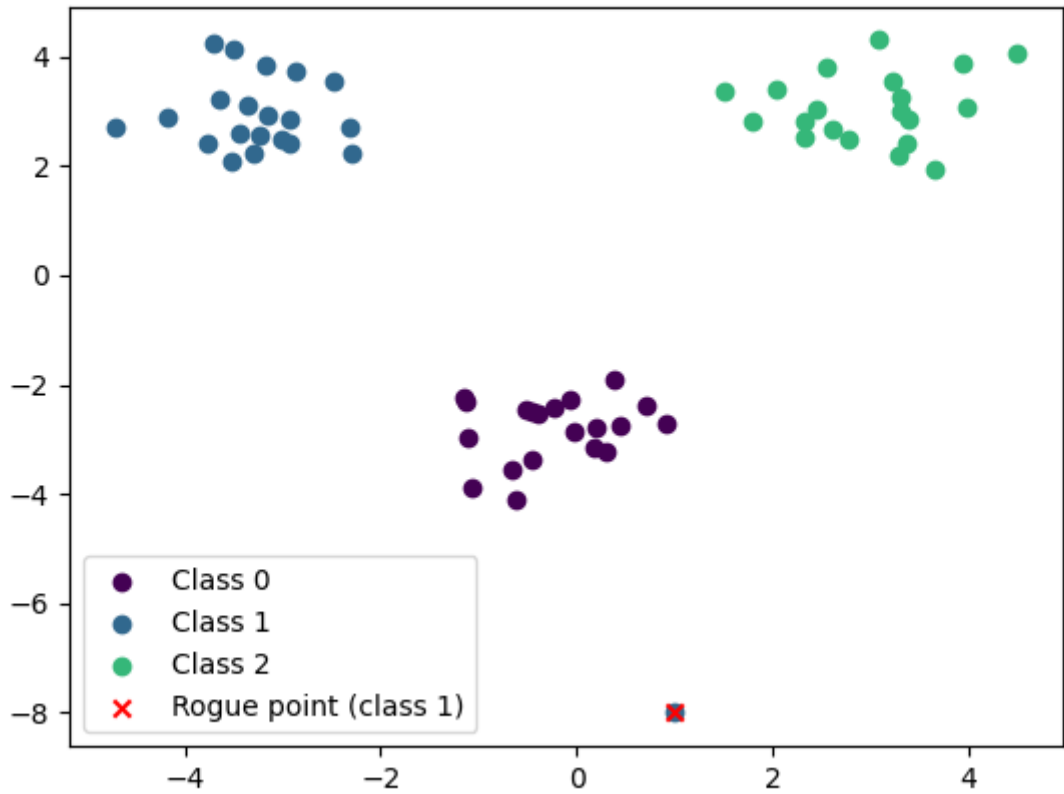
SAE



SISA



5. Rogue point far away from its centroid, but probably in the same decision boundary

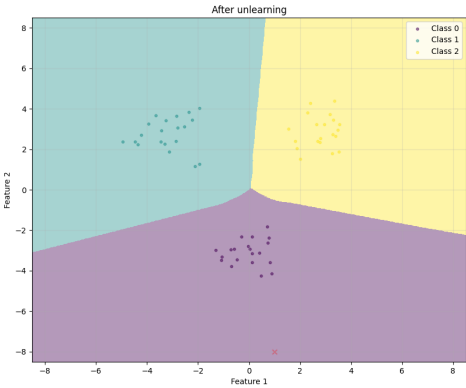
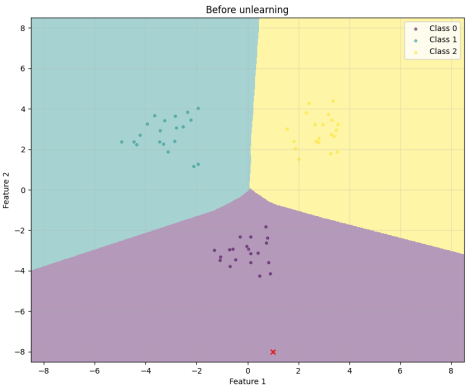


Findings

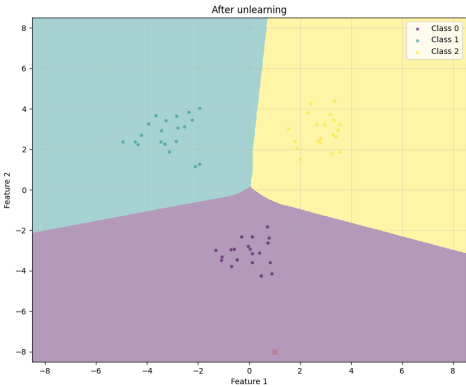
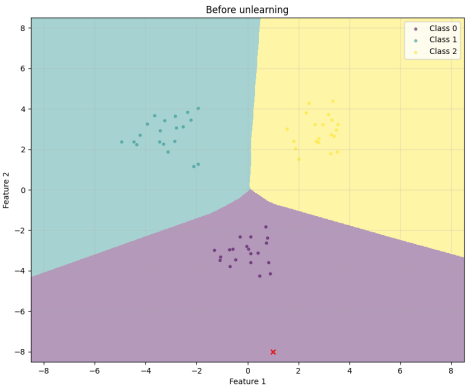
Before unlearning

After unlearning

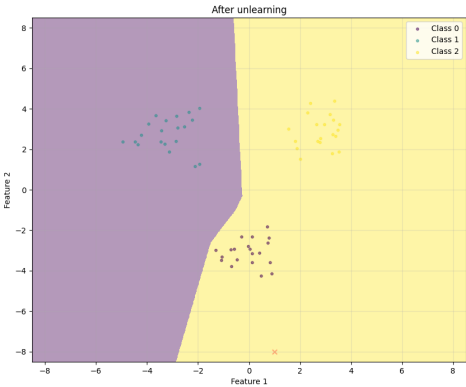
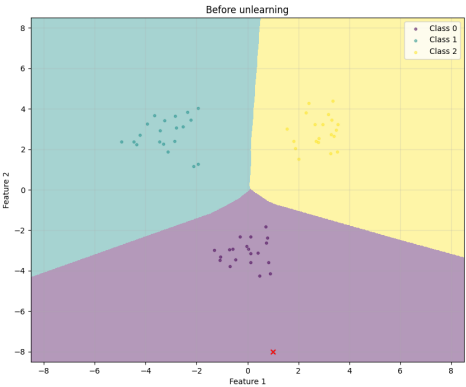
Retrain



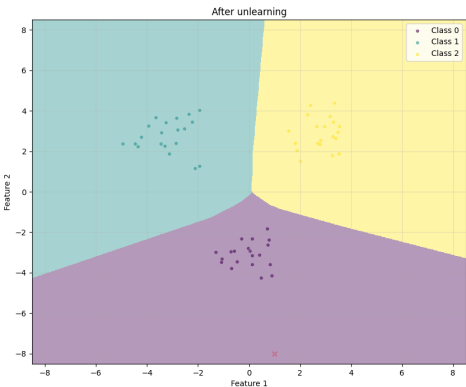
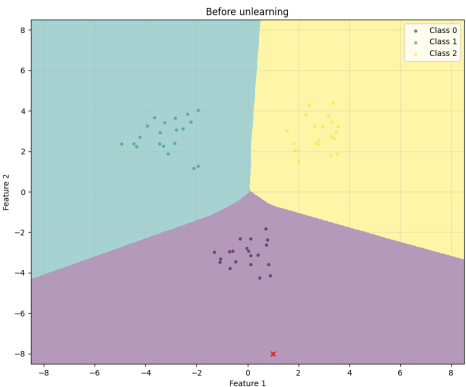
Scrub+R



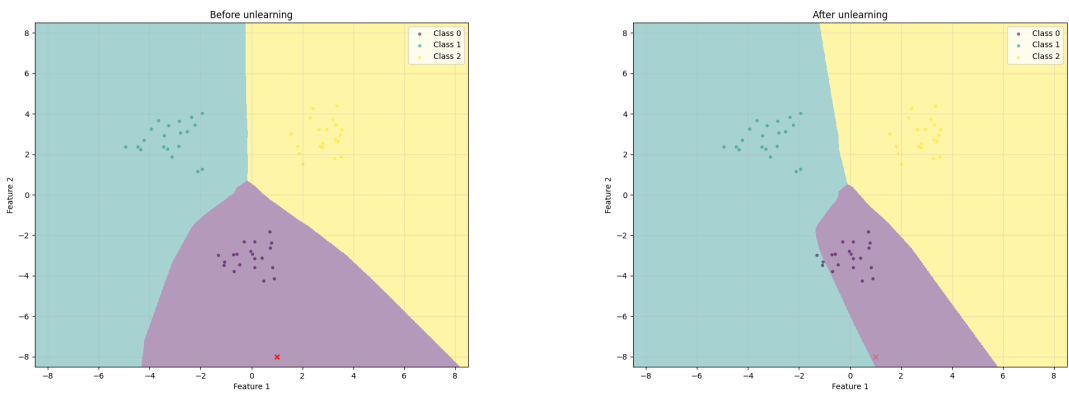
SSD



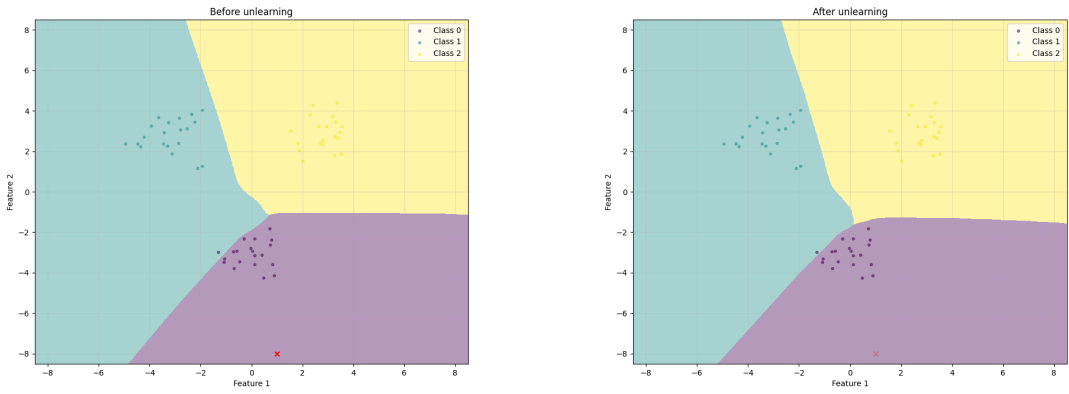
Amnesiac



SAE



SISA



See [Experiment 1: Rogue Many](#) for results with a cluster of rogue points.